PPA-Relevant Clustering-Driven Placement for Large-Scale VLSI Designs

Andrew B. Kahng
UC San Diego
abk@ucsd.edu

Seokhyeong Kang
POSTECH
shkang@postech.ac.kr

Sayak Kundu
UC San Diego
sakundu@ucsd.edu

Kyungjun Min
POSTECH
kj.min@postech.ac.kr

Seonghyeon Park
POSTECH
seonghyeon98@postech.ac.kr

Bodhisatta Pramanik
UC San Diego
bopramanik@ucsd.edu

ABSTRACT

Today's place-and-route (P&R) flows are increasingly challenged by complexity and scale of modern designs. Often, heuristics must trade off between turnaround time and quality of PPA outcomes. This paper presents a clustered placement methodology that improves *both* turnaround time and final-routed solution quality. Our *PPA-aware* clustering considers timing, power and logical hierarchy during netlist clustering, effectively reducing problem size and accelerating global placement runtime while improving post-route PPA metrics. Additionally, our machine learning (ML)-accelerated *virtualized P&R* methodology predicts the best cluster shapes (i.e., aspect ratios and utilizations) to use in P&R of the clustered netlist. With the open-source OpenROAD tool, our methods achieve up to 47% (average: 36%) global placement runtime improvement with similar half-perimeter wirelength (HPWL) and 90% (29%) improvement in post-route total negative slack (TNS). With the commercial Cadence Innovus tool, our methods achieve up to 3.92% (1%) improvement in power and 99% (49%) improvement in TNS.

ACM Reference Format:
Andrew B. Kahng, Seokhyeong Kang, Sayak Kundu, Kyungjun Min, Seonghyeon Park, and Bodhisatta Pramanik. 2024. PPA-Relevant Clustering-Driven Placement for Large-Scale VLSI Designs. In *61st ACM/IEEE Design Automation Conference (DAC '24), June 23–27, 2024, San Francisco, CA, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3649329.3655991

1 INTRODUCTION

The placement phase of physical design is central to optimization of performance, power and area (PPA) outcomes, as well as to design space exploration at floorplan/RTL levels and above. Complexity and scale have rapidly increased, such that millions of instances must be efficiently placed within stringent runtime limits. Thus, placement tools rely on heuristics that are often challenged by problem scale, and by the tension between improvement of turnaround time (TAT) and improvement of quality of results (QOR). *Clustering* has long been seen as a solution to these challenges [20], [11], [9]. However, traditional clustering heuristics [12], [6] only optimize a cutsize criterion and do not consider design information (logical hierarchy, timing, switching activity, etc.) that strongly affects PPA outcomes. Recent works, such as [9] and [14], revisit the use of netlist clustering to guide and improve placement flows. These works demonstrate that clustering can either reduce runtime but with PPA degradation [9], or improve PPA but with runtime degradation [14]. By contrast, our present work develops a *PPA-aware* clustering methodology, and an improved clustered placement approach based on machine learning (ML)-accelerated *virtualized place-and-route* (V-P&R), to improve *both* runtime and PPA relative to baseline (academic and commercial) flat placement methods.

Following are the key contributions made by our work.

- ***PPA-aware* clustering.** We consider additional netlist information – logical hierarchy, timing criticality of paths, and switching activity of nets – to achieve *PPA-aware* clustering. In doing so, we (i) apply a dendrogram-based approach to extract clusters from the netlist logical hierarchy, and (ii) enhance the multilevel clustering framework of [29] [5] to handle logical hierarchy and switching activity of nets. We experimentally demonstrate that our clustering approach achieves noteworthy PPA benefits, and that it outperforms traditional clustering methods when applied in OpenROAD and Cadence Innovus flows (Sections 3.1 and 4).
- **ML-accelerated virtualized P&R.** In the *seeded placement* approach, a *seed placement* of clusters is used to induce *seed locations* of instances, from which the flat P&R flow is continued. Obtaining a high-quality seed placement of clusters requires two elements: how to form the clusters, and how to feed the clusters into a placer. To this end, we use our *PPA-aware* clusters, and a novel V-P&R framework to determine cluster shapes (utilizations and aspect ratios) to use in the cluster placement. We accelerate the V-P&R framework using a graph neural network (GNN)-based ML model that achieves *mean absolute error (MAE)* of 0.131 (for label values in the range [0.564, 2.96]) and *R2 score* of 0.638, (Sections 3.2 and 4.4).

Experimental confirmations. We evaluate our PPA-aware clustering methodology and ML-accelerated V-P&R framework using both OpenROAD and Innovus flows, along with open testcases from the TILOS MacroPlacement [28] and OpenROAD-flow-scripts [27] GitHub repositories. Our methods achieve (maximum, average) percentage improvements of (47, 36) in global placement runtime with similar half-perimeter wirelength (HPWL), compared to the default (i.e., without any clustering or V-P&R) OpenROAD flow. We also achieve (maximum, average) percentage improvements of (4, 1) and (99, 49) in power and post-route total negative slack (TNS), respectively, compared to a standard Innovus flow. To the best of our knowledge, we are the first work to improve *both* global placement runtime and post-route final PPA simultaneously (Section 4).

DAC '24, June 23–27, 2024, San Francisco, CA, USA

ACM ISBN 979-8-4007-0601-1/24/06
https://doi.org/10.1145/3649329.3655991

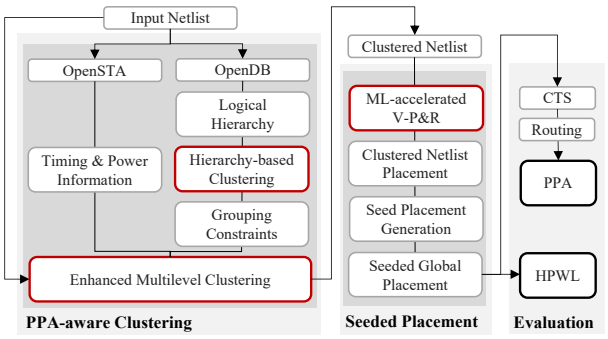


Figure 1: Our overall approach, including *PPA-aware* clustering and the ML-accelerated V-P&R framework.

2 RELATED WORK

Clustering has been widely used in various stages of VLSI physical design, such as partitioning [5], placement [17] and clock tree synthesis [18]. Popular hypergraph clustering heuristics include First Choice (FC) [12], Best Choice (BC) [1] and cut-overlay [6]. FC and BC find clusters of vertices with stronger intra-cluster connectivity compared to inter-cluster connectivity. The cut-overlay method combines multiple clustering solutions to generate better clusters. These methods predominantly rely on local criteria when finding candidate vertices to cluster. By contrast, community detection algorithms such as *Louvain* [4] and *Leiden* [19] adopt a more global perspective, identifying clusters that maximize a *modularity* score.

Several works show benefits of clustering in global placement. [11] proposes a clustering metric that correlates well with post-place wirelength. When applied in clustering-based placement, the metric speeds up placement generation, albeit with some wirelength degradation. [2] integrates BC clustering with placement and demonstrates superior performance over edge-coarsening (EC) and FC. When compared to a standard flow, BC-based placement achieves similar-quality HPWL but with improved placement speed. Similarly, [20] uses clustering in a fast placement methodology. More recently, [9] has proposed a 'blob placement' strategy where placement-relevant clusters are found using Louvain clustering. Integration with RePlAce [7] achieves faster placement runtimes with minor HPWL degradation. Last, the authors of [14] propose an ML-driven clustering framework for PPA optimization. By representing PPA metrics as optimizable loss functions, their method achieves improved PPA results albeit with increased runtime.

With the exception of [14], the above-mentioned works focus on improving global placement runtime. Other gaps are apparent in the literature, e.g., some of the clustering methods used (e.g., BC and Louvain/Leiden) do not scale to large design sizes. Moreover, previous clustering criteria based on cutsize and/or modularity are not well-correlated with PPA outcomes. While [14] proposes a clustering framework that considers additional netlist information for PPA optimization, their approach requires a placed netlist as an input and is runtime-intensive. Our present work addresses these gaps by incorporating PPA-aware clustering and ML-accelerated V-P&R within a clustering-based placement framework that improves both turnaround time and PPA outcomes over strong baseline methods.

3 OUR APPROACH

Our clustering-based placement approach is detailed in Algorithm 1. Figure 1 illustrates the two main components: (i) *PPA-aware* clustering and (ii) ML-accelerated virtualized P&R. The input is a netlist

Algorithm 1: Our overall approach.

Inputs: *Netlist* (.v, .lib, .lef, .def, .sdc), $|P|$ (number of paths), *Tool*
Outputs: *HPWL*, *rWL*, *WNS*, *TNS*, *Power*
1 /* Extract logical hierarchy, timing, and power info */
2 **if** logical hierarchy is present **then**
3 $T(V', E') \leftarrow$ Read logical hierarchy using OpenDB and generate logical hierarchy tree
4 $P \leftarrow$ Extract top $|P|$ timing paths using OpenSTA
5 $S \leftarrow$ Extract switching activity of all nets using OpenSTA
6 $C \leftarrow$ Run hierarchy-based clustering of T using Algorithm 2
7 $Cmty \leftarrow$ Generate grouping constraints based on C
8 /* PPA-aware clustering */
9 $C_{enh} \leftarrow$ Run enhanced multilevel clustering using P, S and $Cmty$
10 $Netlist_{clust} \leftarrow$ Generate clustered netlist from C_{enh}
11 /* ML-accelerated V-P&R */
12 $C_{shapes} \leftarrow$ Generate cluster shapes with ML-accelerated V-P&R for clusters containing more than 200 instances
13 $Netlist_{clust} \leftarrow$ Create .lef using C_{shapes}
14 /* Seeded placement */
15 **if** *Tool* == *Innovus* **then**
16 $Pl_{\text{clust}} \leftarrow$ Run placement on $Netlist_{clust}$
17 Place instances in $Netlist$ at their respective cluster centers
18 Build region constraints using Pl_{clust} and C_{shapes}
19 Run place_design *-incremental*
20 Remove region constraints
21 **else if** *Tool* == *OpenROAD* **then**
22 $Netlist_{clust} \leftarrow$ Scale weights on IO nets by 4
23 Run placement on $Netlist_{clust}$
24 Place instances in $Netlist$ at their respective cluster centers
25 Run globalPlacement *-incremental*
26 /* Placement evaluation */
27 *HPWL* $\leftarrow$ Record HPWL
28 Run CTS and route $Netlist$
29 *rWL*, *WNS*, *TNS*, *Power* $\leftarrow$ Record post-route wirelength, worst negative slack, total negative slack and power from *Tool*
30 **return** *HPWL*, *rWL*, *WNS*, *TNS*, *Power*

file (.v, .lib, .lef, .def[1], .sdc) and a choice of implementation tool (in this work, either *OpenROAD* [25] or *Cadence Innovus* [23]).

***Lines 2-7*:** We use OpenDB [25] to parse the input netlist, extract the logical hierarchy and construct a hierarchy tree T that captures the hierarchical relationship of instances in the netlist. We then use T and Algorithm 2 to find a hierarchy-based clustering of the instances. These clusters are used to induce grouping constraints [5]. We also extract the top P most critical timing paths, along with vectorless switching activity of nets, using OpenSTA [26].

***Lines 9-10*:** *Enhanced multilevel clustering* adds PPA-awareness to the open-source FC implementation of [29]. Hierarchy-based grouping constraints and path timing criticality (slacks) are applied similarly to [5]. We also introduce hyperedge *switching costs* to distinguish nets with high switching activity.[2]

***Lines 12-13*:** Given the clustered netlist, we estimate the best choices of individual cluster shapes (aspect ratios and utilizations) using an ML-accelerated V-P&R framework (Section 3.2). We run ML-accelerated V-P&R only for clusters that contain more than 200 instances.[3] Cluster shapes are then updated in the cluster .lef file.

***Lines 15-25*:** The clustered netlist is placed to obtain a *seed placement*, according to the choice of *Tool*. The coordinates of the seed placement are then used to induce a flat *seeded placement*. When *Tool* is Innovus, we generate the *seeded placement* through a three-step process (Lines 16-20): (i) placing all instances in a cluster at the cluster center; (ii) setting region constraints for clusters whose

[1]The .def file provides floorplan bounding box, pin placements and macro preplacements.
[2]The enhanced FC-based multilevel clustering often generates several singleton clusters. Our background studies show that merging these singleton clusters into a single larger cluster can significantly degrade post-route PPA, hence we do not merge them.
[3]Hyperparameter tuning studies established that a lower bound of 200 instances leads to the best PPA outcomes.

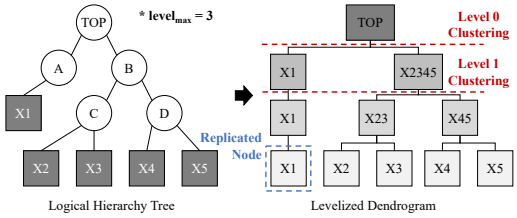


Figure 2: Illustration of hierarchy-based clustering.

shapes were estimated using ML-accelerated V-P&R; and (iii) running incremental placement. When *Tool* is OpenROAD (Lines 22-25), we first scale IO net weights by 4 [9], then generate the *seeded placement* using the above steps (i) and (iii) only, since OpenROAD cannot handle region constraints.

***Lines 27-30*:** To assess quality of the *seeded placement*, we collect post-place wirelength (HPWL), then execute CTS and routing to obtain post-route PPA metrics: routed wirelength (rWL), worst negative slack (WNS), total negative slack (TNS) and total power.

3.1 *PPA-aware* Clustering

Our *PPA-aware* clustering considers logical hierarchy, timing and power information in addition to physical connectivity. Logical hierarchy is obtained using *OpenDB*, while timing and power information is obtained using *OpenSTA*.

Logical hierarchy. Netlist clustering based on logical hierarchy is natural to consider, since functionally 'similar' or 'related' instances are often in spatial proximity in the final placement. However, clustering based only on the logical hierarchy can lead to suboptimalities, since other factors such as timing or connectivity also affect the placement and hence the final PPA. Thus, we use the hierarchy-based clusters as clustering guides (or, grouping constraints), as in [5]. Algorithm 2 formally describes our hierarchy-based clustering; see also Figure 2. Additional details are as follows.

Lines 2-5: We interpret the logical hierarchy tree T as the output of hierarchical clustering and construct a dendrogram, T_{den}, to visualize the hierarchical relationships derived from T (see Figure 2).

Lines 7-12: We *levelize* T_{den} such that all leaf nodes in T_{den} are at the same level (i.e., having the same path distance from the root). The levelization process replicates all leaf nodes that have levels less than $level_{max}$, the largest level of any leaf node of T_{den}. For example, node $x1$ in Figure 2 is replicated once.

Lines 14-24: We evaluate $level_{max} - 1$ clusterings of the netlist, respectively corresponding to the $level_{max} - 1$ levels of T_{den}. Evaluation is according to a weighted average Rent exponent criterion [8], defined by

$$R_{c_i} = \frac{\ln(E(c_i)/(Int(c_i) + Ext(c_i)))}{\ln(|c_i|)} + 1; \quad R_{\text{avg}} = \frac{\sum_{c_i \in C}(R_{c_i} \times |c_i|)}{|V|} \tag{1}$$

Here, R_{c_i} is the Rent exponent for cluster c_i [8]; $E(c_i)$ is total *external* hyperedges (i.e., that connect to vertices in other clusters); $Ext(c_i)$ is total pins in c_i that connect to *external* hyperedges; $Int(c_i)$ is total pins that connect to *internal* hyperedges (i.e., that only connect vertices within c_i); and $|c_i|$ is the number of vertices in c_i. A "good" cluster has a lower value of R_{c_i}. We pick the clustering solution with minimum R_{avg} over all $level_{max} - 1$ clustering solutions.

Timing and power. We extract timing information (top $|P|$ timing-critical paths and net slacks) and net switching activity using the *OpenSTA* tool. We use the *findPathEnds* function from Search.hh available at [26], with group count (|P|) = 100000, endpoint count = 1, unique pins = true, and sort by slack = true. Net switching power is obtained from vectorless power analysis with default tool settings. In particular, we use the *findClkedActivity* function from Sta.hh available at [26]. We leverage the timing information in our PPA-aware clustering by calculating (i) timing cost t_p for critical path p and (ii) timing cost t_e of hyperedge e, as in [5]. We consider power and switching activity by defining a *switching cost* of hyperedge e:

$$s_e = (1 + \frac{\theta_e}{\sum_{e \in E} \theta_e})^{\mu} \tag{2}$$

where θ_e is the switching activity of a hyperedge e and μ (default=2) is a scaling factor. The *heavy-edge rating function* of [5] is then extended as:

$$r_{\text{overall}}(u, v) = \sum_{e \in I(v) \cap I(u)} \frac{\langle \alpha, w_e \rangle + \beta t_e + \gamma s_e}{|e| - 1} \tag{3}$$

where u and v denote the pair of cluster candidates while $I(v)$ denotes the incident hyperedges of v. The parameters α, β and γ are scaling factors.

Algorithm 2: Hierarchy-based clustering.

Inputs: Hypergraph $H(V, E)$, Logical hierarchy tree $T(V', E')$
Output: Cluster assignments C

1 /* Construct the dendrogram */
2 $T_{den}(V_{den}, E_{den}) \leftarrow$ Initialize dendrogram using T
3 $leaf\_vertices \leftarrow$ Leaf vertices in T_{den}
4 $levels \leftarrow$ Generate levels of each node in T_{den}
5 $level_{\max} \leftarrow \max(levels)$
6 /* Levelizing the dendrogram by replicating leaf nodes */
7 **for** *each* $v \in V_{den}$ **do**
8 **if** v *is a leaf node and* $level(v) < level_{max}$ **then**
9 **for** $k \leftarrow level(v); k < level_{max}; k \leftarrow k + 1$ **do**
10 $v_{\text{copy}} \leftarrow$ Create a copy of node v
11 Assign v_{copy} to be v's child
12 $v \leftarrow v_{\text{copy}}$
13 /* Find best clustering */
14 $C \leftarrow$ Initialize array of cluster assignments for V
15 $R_{avg_{best}} \leftarrow \infty$
16 **for** $k \leftarrow 0; k < level_{max} - 1; k \leftarrow k + 1$ **do**
17 $V_{den_k} \leftarrow$ All vertices in V_{den} that have level k
18 $C_k \leftarrow$ Clustering solution at level k
19 $R_{avg_k} \leftarrow$ Weighted average of Rent's parameter (Equation 1)
20 **if** $R_{avg_k} < R_{avg_{best}}$ **then**
21 $R_{avg_{best}} \leftarrow R_{avg_k}$
22 $C \leftarrow C_k$
23 /* Return best clustering */
24 **return** C

3.2 V-P&R and ML-based *Acceleration*

Cluster shapes (utilization and aspect ratio) significantly impact seed placement and PPA outcomes, as documented in Section 4.4 below. We therefore introduce a *virtualized P&R* (V-P&R) framework to determine the best shape for each cluster. We further apply GNN-based ML modeling to accelerate the V-P&R framework.

Virtualized P&R. The basic idea of V-P&R is that by running place-and-route on the sub-netlist induced by a cluster, we can gain insight into how to model that cluster during seed placement. Figure 3 shows our V-P&R framework. For each given cluster, we first induce the sub-netlist over the instances in the cluster. For each inter-cluster net that is incident to the given cluster, we create input (output) ports in the sub-netlist, corresponding to any sinks (driver) in the cluster. This sub-netlist is passed to the V-P&R framework, along with 20 different combinations of 5 aspect ratios and 4 utilizations. For each combination of aspect ratio and utilization,

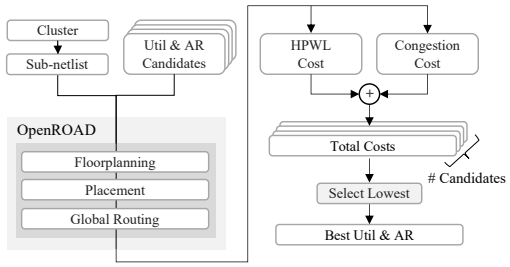


Figure 3: Virtualized P&R flow.

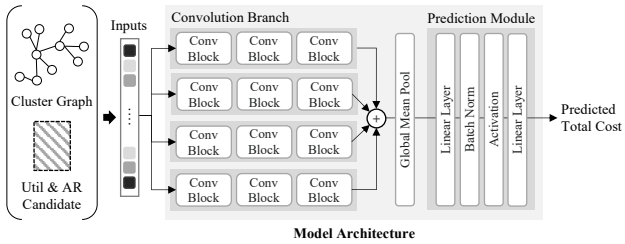


Figure 4: GNN-based architecture to predict *Total Cost*.

we initialize the floorplan of a "virtual die", then run placement and global routing on the sub-netlist using the default OpenROAD flow script [27]. We then record HPWL and routing congestion.[4] To identify a combination of aspect ratio and utilization that achieves both good HPWL and low congestion, we define *HPWL Cost* as

$$Cost_{HPWL} = \frac{HPWL_{avg}}{Width_{Core} + Height_{Core}} \quad (4)$$

where $HPWL_{avg}$ is the average HPWL of nets in the sub-netlist, and $Width_{Core}$ and $Height_{Core}$ are respectively the width and height of the virtual die. We also define *Congestion Cost* as

$$Cost_{Congestion} = \frac{\sum^{TopX\%\ GCells} Congestion}{TopX\%\ GCells} \quad (5)$$

where X is a hyperparameter (default = 10). Following [13], we define overall *Total Cost* as: $Total\ Cost = Cost_{HPWL} + \delta * Cost_{Congestion}$, where δ is a normalization factor (default = 0.01). The (aspect ratio, utilization) combination that achieves best *Total Cost* is used to create the cluster's .lef model during seed placement.

ML Modeling to Accelerate V-P&R. As described above, V-P&R determines each cluster's ideal shape by running OpenROAD 20 times through the end of global routing (each run can require as much as 3 seconds). This effort grows linearly with design size, and can reach undesirable levels. To address this, we implement an ML-based strategy that in practice accelerates our V-P&R framework by approximately 30×. Specifically, we use a GNN-based model to predict *Total Cost*, replacing execution of OpenROAD in Figure 3.

Our ML model training uses a diversity of clusters generated by perturbing seed and coarsening hyperparameters [29] in our PPA-aware clustering. The training, validation and testing datasets respectively consist of 22700, 5600 and 3200 clusters. Following [9], we sweep aspect ratio in the range [0.75, 1.75] with step size 0.25,[5] and sweep utilization in the range [0.75, 0.90] with step size 0.05. This results in 20 distinct (aspect ratio, utilization) combinations, i.e., 20 candidates for the cluster shape. For each candidate, we run (i) V-P&R, (ii) calculate *Total Cost*, and (iii) use *Total Cost* as a label.

Figure 4 shows our GNN-based model architecture. We convert a cluster's sub-netlist to an undirected graph ("Cluster Graph") using standard clique expansion with edge weight $1/(|e| - 1)$ for each hyperedge e [16]. Each node in the graph has 28 features, extending [15] with two new features italicized below. These features are:

- **Design parameters:** floorplan utilization and aspect ratio.
- **Cluster-level features:** #cells, #nets, #pins, #nets w/ fanout 5-10, #nets w/ fanout > 10, #internal nets, #border nets, total cell area, average cell degree, *average net degree*, average clustering coefficient, density, diameter, radius, edge connectivity, #colors for greedy coloring and average global efficiency.
- **Cell-level features:** Cell area, cell degree, average neighborhood degree, betweenness centrality, closeness centrality, *degree centrality* [21], clustering coefficient, eccentricity and cell type.

Our model takes as input a cluster graph and a candidate shape. Structurally, our model comprises four distinct convolution branches, with each branch containing three convolution blocks (Figure 4). Dimensions of the input, hidden and output layers of the convolution branches are 35, 64 and 32, respectively. The convolution blocks are implemented with hypergraph convolution [3], batch normalization, and skip connections that are specifically used when the input and output dimensions match. As the input data passes through each convolution branch, the results from these branches are accumulated; then, global mean pooling is applied to generate an embedding vector for the cluster. This vector is subsequently passed to a prediction module that predicts the *Total Cost*. Our prediction module is built with two linear layers, batch normalization and an activation function (Figure 4). Here, dimensions of the input, hidden and output layers are 32, 64 and 1, respectively. After predicting the *Total Cost* for all candidates, the candidate with the lowest *Total Cost* is returned as the output of the ML-accelerated V-P&R framework. (Full details and scripts can be seen at [22].)

4 EXPERIMENTAL EVALUATION

Our PPA-aware clustering framework is written in C++ and built on the OpenROAD infrastructure. Our ML model is implemented using *PyTorch Geometric*. We make available all codes and scripts at our GitHub repository [22]. We run all experiments on a server with four 2.4 GHz Intel Xeon(R) Gold 6148 processors and 376 GB RAM. For evaluation we use testcases that are publicly available in the MacroPlacement [28] and OpenROAD [27] GitHub repositories. We use six designs (aes, ariane, BlackParrot, jpeg, MegaBoom and MemPool Group) and the NanGate45 [24] open enablement in our experiments. Table 1 lists the main statistics of these benchmarks. We evaluate our PPA-aware clustering and ML-accelerated V-P&R with OpenROAD and Innovus *v.21.1*.[6] For clarity, we divide our validation efforts into two categories: (i) validation of runtime and PPA with OpenROAD (Section 4.1) and (ii) validation of PPA with Innovus (Section 4.2). To show the benefits of our PPA-aware clustering and ML-accelerated V-P&R methods, we present ablation studies in Sections 4.3 and 4.4. Finally, we explore the tuning of hyperparameters in Section 4.5.

4.1 PPA and Runtime Validation (OpenROAD)

We evaluate our PPA-aware clustering and ML-accelerated V-P&R methods using OpenROAD. We compare our post-place HPWL and

[4] Based on aspect ratio and utilization, a floorplan .lef is created for P&R of the sub-netlist. The IO ports are placed with the OpenROAD pin placer; in the Nangate45 enablement these use the *metal2* (vertical) and *metal3* (horizontal) layers.

[5] More extreme cluster aspect ratios (< 0.75 or > 1.75) generally result in poor PPA.

[6] We do not perform any benchmarking of commercial EDA tools. Further, to avoid inadvertent benchmarking, we mask the target clock period values TCP_{Inv} in Table 1.

Table 1: Specifications of benchmarks.

Design (NG45)	#Insts	#Nets	*TCP$_{OR}$	*TCP$_{Inv}$
aes	15547	16338	0.55	-
jpeg	53042	58898	0.80	-
ariane	119256	142226	1.80	-
BlackParrot (BP)	768851	998716	2.30	-
MegaBoom (MB)	1086920	1443755	NA	-
MemPool Group (MP-G)	2729729	3087191	NA	-

*TCP$_{OR}$ (ns) and TCP$_{Inv}$ denote the target clock periods used in the OpenROAD and Innovus flows, respectively. TCP$_{Inv}$ is masked to avoid inadvertent benchmarking of the Innovus tool.

runtime with [9] and the default OpenROAD flow [27] in Table 2. We do not compare with [14] as code was not available from the authors. We first report the cumulative runtimes of clustering and seeded placement (for [9] and our approach) and then normalize these numbers by the placement runtime recorded from the default flow. We observe that our methods achieve up to: (i) 60% improvement in runtime and 5% improvement in HPWL compared to [9] and (ii) 47% improvement in runtime and 1% improvement in post-place HPWL. For MegaBoom and MemPool Group, the clustering runtime of [9] is significantly larger (~2X) than the placement runtime of OpenROAD. We hence omit these with "NA" (in Table 2). We separately give the runtime breakdown of our approach in [22]. We also measure the post-route PPA in Table 3. Here, we exclude MegaBoom and MemPool Group since OpenROAD fails to route for these designs. In Table 3, we exclude [9] in our post-route PPA, since we evaluate our method using a superior community detection algorithm (Leiden) in Section 4.3. We observe that our methods achieve maximum (average) percentage PPA improvements of 5 (2), 63 (26), 90 (29) and 0.7 (0.2) in *rWL*, *WNS*, *TNS* and *Power*, respectively, when compared to the default flow. We observe marginal improvement in power compared to the default flow. However, when compared to clustering methods with no PPA-awareness, our method achieves up to 5% improvement in power (Section 4.3).

Table 2: Evaluation of post-place results with OpenROAD.

Design	[9]		Ours	
	HPWL	CPU	HPWL	CPU
aes	1.007	1.567	**1.000**	**0.802**
jpeg	**0.973**	1.000	1.011	**0.524**
ariane	1.046	0.931	**0.989**	**0.604**
BlackParrot	0.996	1.824	**0.989**	**0.738**
MegaBoom	NA	NA	**0.997**	**0.664**
MemPool Group	NA	NA	1.001	**0.533**

Table 3: Evaluation of post-route results with OpenROAD.

Design	Flow	Post-route PPA			
		rWL	WNS	TNS	Power
aes	*Default*	1.00	-220	-32.08	0.296
	Ours	**0.98**	**-210**	**-31.85**	**0.294**
jpeg	*Default*	1.00	**-410**	**-130.45**	0.441
	Ours	**0.95**	-470	-177.73	**0.438**
ariane	*Default*	1.00	-200	-139.21	**0.655**
	Ours	**0.99**	**-100**	**-14.21**	0.661
BP	*Default*	1.00	-410	-1441.24	4.93
	Ours	**0.99**	**-150**	**-563.69**	**4.91**

The unit of WNS is *ps*, the unit of TNS is *ns*, and the unit of Power is *W*.

4.2 PPA Validation (Cadence Innovus)

In this section, we validate our PPA-aware clustering and ML-accelerated V-P&R methods with Cadence Innovus *v.21.1*. Table 4 compares post-route PPA metrics to those obtained with the standard Innovus flow [28]. For all designs, our methods significantly improve most PPA metrics. We achieve maximum (average) percentage improvements of 1.9 (0.2), 98 (35), 99 (49) and 4 (1) in *rWL*, *WNS*, *TNS* and *Power*, respectively. We observe similar runtime compared to the standard Innovus flow.

Table 4: Evaluation of post-route results with Innovus.

Design	Flow	Post-route PPA			
		rWL	WNS	TNS	Power
aes	*Default*	**1.000**	-72	-7.94	**0.050**
	Ours	1.014	**-60**	**-7.37**	**0.050**
jpeg	*Default*	**1.000**	-41	-1.17	0.284
	Ours	1.005	**-6**	**-0.06**	**0.273**
ariane	*Default*	1.000	-97	-52.73	**0.841**
	Ours	**0.998**	**-80**	**-39.86**	0.842
BP	*Default*	1.000	**-134**	-548.74	4.492
	Ours	**0.999**	-165	**-372.80**	**4.481**
MB	*Default*	1.000	-28	-1.34	1.611
	Ours	**0.981**	**-5**	**-0.014**	**1.586**
MP-G	*Default*	1.000	-37	-0.96	**2.671**
	Ours	**0.994**	**-29**	**-0.63**	2.679

The unit of WNS is *ps*, the unit of TNS is *ns*, and the unit of Power is *W*.

4.3 Comparison of PPA-awareness

We now assess the PPA-relevance of our clustering method, relative to Leiden clustering and TritonPart's default clustering method (multilevel FC, denoted as MFC in Table 5). We use post-route PPA metrics obtained from using Leiden and multilevel FC in our overall flow. Table 5 presents evaluation with OpenROAD (more results are available in [22]). Routed wirelength is normalized to the value obtained with the default OpenROAD flow. We observe that compared to Leiden, our clustering approach leads to better PPA outcomes – up to 5% improvement in *rWL*, *WNS* and *TNS* and up to 2% improvement in *Power*. Compared to the multilevel FC, our clustering achieves better percentage PPA improvements (up to 6, 13, 10 and 2, respectively) on the same metrics. These results indicate that consideration of additional netlist information (logical hierarchy, timing path slacks, and switching activity of nets) during clustering helps to improve the final PPA. Thus, the Table 5 data confirm PPA-relevance of our proposed clustering methodology.

Table 5: Evaluation of our PPA-aware clustering framework.

Design	Method	Post-route PPA			
		rWL	WNS	TNS	Power
aes	*Leiden*	0.991	-211	-33.41	0.295
	MFC	1.028	-214	-42.42	0.311
	Ours	**0.980**	**-210**	**-31.85**	**0.294**
jpeg	*Leiden*	0.998	-472	-177.82	0.446
	MFC	1.011	-542	-196.42	0.459
	Ours	**0.950**	**-470**	**-177.73**	**0.438**
ariane	*Leiden*	0.996	-105	-14.29	0.672
	MFC	1.001	-134	-16.28	0.669
	Ours	**0.972**	**-100**	**-14.21**	**0.661**

The unit of WNS is *ps*, the unit of TNS is *ns*, and the unit of Power is *W*.

4.4 V-P&R Model Evaluation

We evaluate the performance of our GNN-based model using two metrics: (i) mean absolute error (*MAE*), which evaluates the average magnitude of absolute prediction errors, aid (ii) *R2 score*, which quantifies the amount of variance in the predicted values. The values of the (*Total Cost*) labels lie in the range [0.564, 2.96] and have a mean of 1.703 with standard deviation 0.727. Our results show that we achieve an *MAE* of 0.105, 0.113, and 0.131 for the training, validation, and test datasets, respectively. Our *R2 score* is 0.788, 0.753, and 0.638 for the three datasets. These metrics confirm the model prediction accuracy of our GNN-based architecture.

Table 6 presents PPA benefits of our ML-accelerated V-P&R framework, using Innovus, again with more results available in [22]. In this study, we first substitute the ML-accelerated V-P&R framework (denoted as *V-P&R$_{ML}$* in the table) with (i) random cluster shape assignments (*Random*) or with (ii) fixed cluster shape assignments where each cluster is assigned utilization = 0.9 and

Table 6: Evaluation of our ML-based V-P&R framework.

Design	Shape	Post-route PPA			
		rWL	WNS	TNS	Power
ariane	*Random*	0.992	-94	-59.80	**0.840**
	Uniform	1.000	-103	-66.12	**0.840**
	V-P&R$_{ML}$	**0.977**	**-80**	**-39.86**	0.842
jpeg	*Random*	1.010	-11	-0.29	0.288
	Uniform	1.000	-19	-0.33	0.279
	V-P&R$_{ML}$	**0.996**	**-6**	**-0.06**	**0.273**
MB	*Random*	0.999	-17	-0.71	1.596
	Uniform	1.000	-14	-0.40	1.601
	V-P&R$_{ML}$	**0.961**	**-5**	**-0.014**	**1.586**

The unit of WNS is ps, the unit of TNS is ns, and the unit of Power is W.

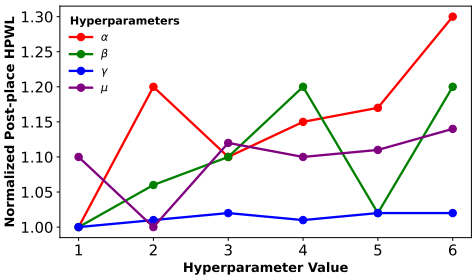


Figure 5: Hyperparameter validation.

aspect ratio = 1.0 (*Uniform*). Then, we run the clustering-driven placement and collect the post-route PPA metrics. In Table 6, all *rWL* values for each design are normalized to the value obtained with uniform cluster shape assignments. The results show that compared to (random, uniform) assignments, ML-accelerated V-P&R achieves (arithmetic) average percentage improvements of $(2, 2)$, $(44, 52)$, $(85, 73)$ and $(2, 1)$ in *rWL*, *WNS*, *TNS* and *Power*, respectively,

4.5 Hyperparameter Selection

We have determined default values for the hyperparameters (α, β, γ and μ) by performing a study involving three designs: *aes*, *jpeg*, and *ariane*. We define the score value as the arithmetic mean of the post-place HPWL improvement.[7] In our experiments, for each design, we: (i) vary each parameter while keeping the other three constant[8]; (ii) run our flow with OpenROAD and record the post-place HPWL; and (iii) normalize the HPWL to the value obtained with our default hyperparameter settings (see Figure 5). From our findings, we observe that our default setting of the hyperparameters is a reasonable choice. A more detailed exploration of the hyperparameter space – with more designs and impact on post-route PPA– is provided in our GitHub repository [22].

5 CONCLUSION

We have developed new *PPA-aware clustering* and ML-accelerated *virtualized P&R* methods that improve *seed placements* for large-scale global placement. For PPA-aware clustering, we adapt the multilevel clustering framework of [29] to consider logical hierarchy, timing paths and switching activities. Our ML-accelerated V-P&R framework efficiently predicts beneficial aspect ratios and utilizations to apply with clusters produced by the PPA-aware clustering. As noted above, our ML-model accelerates the V-P&R framework by 30× with a one-time training cost. Together, these elements enable generation of a high-quality *seed placement* that leads to final placements with improved post-route PPA metrics. Experimental results confirm both PPA *and* runtime benefits of our methods. When integrated with the open-source OpenROAD tool, our methods can improve *both* PPA and runtime, with 29% average TNS improvement and 36% runtime speedup. With the commercial Cadence Innovus tool, we achieve better PPA, with 49% average TNS improvement. Our ongoing research pursues confirmation of the benefits from our methods on additional testcases, design enablements and P&R tools. We are also studying the effects of different cluster shapes (L-shaped, diamond, circle, etc.) on placement, and enhancing power-awareness of our clustering methodology to further improve the post-route power metric. Last, we plan to study the benefits of our *PPA-aware* clustering and ML-accelerated V-P&R framework in the context of 3D placement.

[7] We consider HPWL for faster TAT and more extensive parameter space exploration.
[8] The x-axis in Figure 5 denotes multipliers applied to the default values of the hyperparameters. Our study assesses whether there are benefits to larger relative weighting of the terms in Equation 3. Hence we sweep our multipliers of default values in the range $[1, 6]$ with step size 1.

ACKNOWLEDGMENTS

We thank the authors of [9] for sharing scripts and codes to run their flows. We also thank the authors of [15] for sharing their feature extraction codes and ML model. This work is supported in part by DARPA HR0011-18-2-0032 (OpenROAD), the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korean government (MSIT) (No. 2021-0-00754, Software Systems for AI Semiconductor Design) and by the Samsung Advanced Institute of Technology.

REFERENCES

[1] C. J. Alpert, A. B. Kahng, G.-J. Nam, S. Reda and P. Villarrubia, "A Semi-Persistent Clustering Technique for VLSI Circuit Placement", *Proc. ISPD*, 2005, pp. 200–207.
[2] C. Alpert, A. Kahng, G.-J. Nam, S. Reda and P. Villarrubia, "A Fast Hierarchical Quadratic Placement Algorithm", *IEEE Trans. CAD* 25(4) (2006), pp. 678–691.
[3] S. Bai, F. Zhang and P. H. S. Torr, "Hypergraph Convolution And Hypergraph Attention", *Pattern Recognition*, 110 (2021), 107637.
[4] V. D. Blondel, J. L. Guillaume, R. Lambiotte and E. Lefebvre, "Fast Unfolding of Communities in Large Networks", *J. Stat. Mech. Theor. Exp.*, 10 (2008), pp. 1-12.
[5] I. Bustany, G. Gasparyan, A. B. Kahng, Y. Koutis, B. Pramanik and Z. Wang, "An Open-Source Constraints-Driven General Partitioning Multi-Tool for VLSI Physical Design", *Proc. ICCAD*, 2023, pp. 1-9.
[6] I. Bustany, A. B. Kahng, Y. Koutis, B. Pramanik and Z. Wang, "SpecPart: A Supervised Spectral Framework for Hypergraph Partitioning Solution Improvement", *Proc. ICCAD*, 2022, pp. 1-9.
[7] C.-K. Cheng, A. B. Kahng, I. Kang and L. Wang, "RePlAce: Advancing Solution Quality and Routability Validation in Global Placement", *IEEE Trans. CAD* 38(9) (2019), pp. 1717-1730.
[8] Y. Cheon and D. F. Wong, "Design Hierarchy Guided Multilevel Circuit Partitioning", *Proc. ISPD*, 2002, pp. 30-35.
[9] M. Fogaça, A. B. Kahng, E. Monteiro, R. Reis, L. Wang and M. Woo, "On the Superiority of Modularity-Based Clustering for Determining Placement-Relevant Clusters", *Integration: The VLSI Journal* 74 (2020), pp. 32-44.
[10] L. C. Freeman, "Centrality in Social Networks Conceptual Clarification", *Social Networks* 1(3) (1978) pp. 215-239.
[11] B. Hu and M. Marek-Sadowska, "Fine Granularity Clustering-Based Placement", *IEEE Trans. CAD* 23(4) (2004), pp. 527-536.
[12] G. Karypis and V. Kumar, "Multilevel K-Way Hypergraph Partitioning", *Proc. DAC*, 1999, pp. 343-348.
[13] M.-C. Kim, N. Viswanathan, C. J. Alpert, I. L. Markov and S. Ramji, "MAPLE: Multilevel Adaptive Placement For Mixed-size Designs", *Proc. ISPD*, 2012.
[14] Y.-C. Lu, T. Yang, S. K. Lim and H. Ren, "Placement Optimization Via PPA-Directed Graph Clustering", *Proc. MLCAD*, 2022, pp. 1-6.
[15] K. Min, S. Kwon, S.-Y. Lee, D. Kim, S. Park and S. Kang, "ClusterNet: Routing Congestion Prediction and Optimization Using Netlist Clustering and Graph Neural Networks", *Proc. ICCAD*, 2023, pp. 1-9.
[16] T. Lengauer, *Combinatorial Algorithms For Integrated Circuit Layout*, Wiley Teubner, 1990.
[17] J. A. Roy, S. N. Adya, D. A. Papa and I. L. Markov, "Min-cut Floorplacement", *IEEE Trans. CAD* 25(7) (2006), pp. 1313—1326.
[18] R. S. Shelar, "An Efficient Clustering Algorithm for Low Power Clock Tree Synthesis", *Proc. ISPD*, 2007, pp. 181–188.
[19] V. A. Traag, L. Waltman and N. J. van Eck, "From Louvain To Leiden: Guaranteeing Well-Connected Communities", *Scientific Reports* 9 (2019).
[20] N. Viswanathan, M. Pan and C. Chu, "FastPlace 3.0: A Fast Multilevel Quadratic Placement Algorithm With Placement Congestion Control", *Proc. ASPDAC*, 2007, pp. 135-140.
[21] J. Zhang and Y. Luo, "Degree Centrality, Betweenness Centrality, and Closeness Centrality in Social Network", *Intl. Conf. on MSAM*, 2017, pp. 300–303.
[22] ABKGroup/BlobPlacement GitHub Repository. https://github.com/ABKGroup/BlobPlacement
[23] Cadence Innovus Implementation System, version 21.1. www.cadence.com
[24] NanGate45 PDK. https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts/tree/master/flow/platforms/nangate45
[25] The OpenROAD Project (GitHub). https://theopenroadproject.org and https://github.com/The-OpenROAD-Project/OpenROAD
[26] OpenSTA Static Timer. https://github.com/The-OpenROAD-Project/OpenSTA
[27] OpenROAD-Flow-Scripts. https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts
[28] TILOS MacroPlacement repository. https://github.com/TILOS-AI-Institute/MacroPlacement
[29] TritonPart: An Open-Source Constraints-Driven Partitioner. https://github.com/The-OpenROAD-Project/OpenROAD/tree/master/src/par

OPTIMA: Design-Space Exploration ofDischarge-Based In-SRAM Computing: Quantifying Energy-Accuracy Trade-offs

*Saeed Seyedfaraji, **Severin Jäger, *Salar Shakibhamedan, *Asad Aftab, +Semeen Rehman
*,**,+Technische Universität Wien (TU Wien), Vienna, Austria
+University of Amsterdam, The Netherlands
*{saeed.seyedfaraji, salar.shakibhamedan, asad.aftab}@tuwien.ac.at,**e1613004@student.tuwien.ac.at,+s.rehman@uva.nl

***Abstract*—In-SRAM computing promises energy efficiency, but circuit nonlinearities and PVT variations pose major challenges in designing robust accelerators. To address this, we introduce OPTIMA, a modeling framework that aids in analyzing bit-line discharge and power consumption in 6T-SRAM-based accelerators. It provides insights into limiting factors and enables fast design-space exploration of circuit configurations. Leveraging OPTIMA for in-SRAM multiplications exhibits ∼100 × simulation speed-up while maintaining an RMS modeling error of 0.88 mV. Exploration yields an optimized multiplier with 1.05 pJ energy consumption per 4-bit operation and classification accuracies of 71.8 % (top-1) and 90.4 % (top-5) for ImageNet and 92.5 % for CIFAR-10 datasets respectively when applied in quantized DNNs. To further support research and development efforts, we have made our OPTIMAptima tool flow available as open source [https://github.com/sevjaeg/optima].**

***Index Terms*—processing-in-memory, in-memory computing, SRAM, Deep Neural Networks, Image Classification, Open source**

I. Introduction

Over recent decades, semiconductor scaling has significantly advanced computing performance. Despite the exponential growth in transistors, modern processors face limitations in power consumption, operating frequency, and single-thread performance [1]. The memory wall [2] and the power wall [3] constrain applications not by processor throughput but by energy consumption and memory system performance. Although complex memory hierarchies mask these issues, workloads like Machine Learning (ML) and big data suffer from high memory latencies, causing processor idle time and performance degradation [4].

In systems-on-chip, power is not only consumed by the computational logic, but also in the memory system, for example in Static Random Access Memory (SRAM) leakage power and off-chip interconnects for Dynamic Random Access Memory (DRAM). Data movement between memory and computation units dominates, accounting for over 60% of the total energy use [5]. New computing paradigms based on emerging non-volatile memory or vertical memory integration show potential in reducing energy per data transfer or off-chip communication energy [6], [7]. However, minimizing data movement requires innovative paradigms like Near-Memory Computing (NMC) or In-Memory Computation (IMC). NMC adds computational cores close to the memory system, while IMC enhances the memory itself with computational capabilities. This minimizes data movement and yields optimal energy efficiency. This work specifically explores IMC.

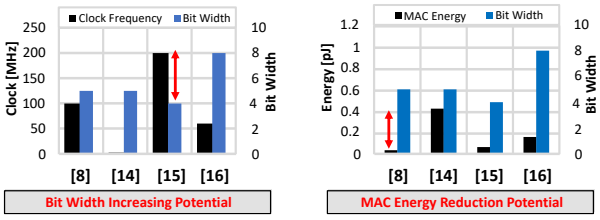


Fig. 1: State-of-the-art in-SRAM multiplication design space

IMC targets energy-efficient computation by reducing memory transfer time and energy for tasks like ML inference and operations such as Multiplication and Accumulation (MAC) and matrix-vector multiplication. Leveraging the existing cache area of modern Central Processing Units (CPUs) for SRAM-based IMC can enhance energy efficiency without significant architectural changes. Prior to incorporating IMC methodologies, it is crucial to initially discern the attributes of the diverse alternatives. Within the array of memory technologies employed for IMC, 6T SRAM presents notable sub-bank divisibility (enabling parallel computing), convenient accessibility, and economical fabrication costs [8], [9]. Hence, 6T SRAM becomes a promising candidate for IMC.

In-SRAM computing has two major flavors: bit-line (BL) computing and discharge-based computing. The former relies on concurrent activation of word lines (WLs) and logic primitives in readout amplifiers, often augmented with digital NMC logic. This approach accelerates various applications, including search [10], floating-point arithmetic [11], and neural networks [12]. On the other hand, discharge-based computing utilizes the analog nature of read-out transistors to create a data-dependent discharge on SRAM's BLs. This allows operations like addition [13] or multiplication [8] directly in the memory array, promising even greater energy savings.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

ACM ISBN 979-8-4007-0601-1/24/06.
https://doi.org/10.1145/3649329.3661852

The authors gratefully acknowledge funding from European Union's Horizon 2020 Research and Innovation programme under the Marie Skłodowska Curie grant agreement No. 956090 (APROPOS: Approximate Computing for Power and Energy Optimisation, http://www.apropos.eu/).

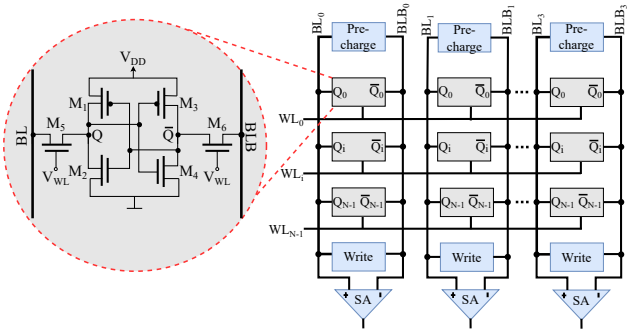


Fig. 2: 6T SRAM cell and SRAM array

Thus, this work emphasizes discharge-based SRAM IMC.

A. Problem Motivation and Research Challenge

Discharge-based in-SRAM computing circuits are typically optimized for energy consumption, latency, accuracy, or area resulting in a rich design space. Fig. 1 compares different design points for IMC multipliers [8], [14]–[16]. Clearly, there are significant trade-offs between the design metrics. For instance, the circuit presented in [16] shows higher bit widths thus, offering high accuracy, whereas the other circuits demonstrated higher latency thus, they are significantly slower compared to the multiplier demonstrated in [15].

These trade-offs have to be investigated thoroughly with design-space exploration to find (Pareto-)optimal configurations. However, the analog nature of these circuits leads to large configuration spaces which are typically evaluated with slow circuit simulations based on solving differential equations. In addition, Process, Voltage, and Temperature (PVT) variations have to be considered, adding significant run-time overhead. Therefore, the selection of an optimal analog-based IMC configuration is very time-consuming, thus inefficient, and needs to be addressed.

B. Our Novel Contribution

To address the above-discussed research challenge, we propose a modeling technique called OPTIMA. This manuscript's contributions are as follows:

- In the first step, we identify and analyze the error sources at the circuit level and assess their impact on SRAM IMC circuits in Section III.
- Subsequently, in Section IV we develop a precise model for SRAM BL discharge and power consumption, considering nonlinearities and PVT variations. To validate our model, we utilize circuit simulation data from a 65 nm technology transistor model.
- In Section V, we showcase OPTIMA by performing fast design-space exploration of a 4-bit multiplication circuit. We derive a circuit configuration with an optimized energy-accuracy tradeoff.
- Finally, we assess the effectiveness of the optimized in-memory multiplier in Deep Neural Networks (DNNs). In Section VI, we evaluate the accuracy with four standard networks on two major data sets.

II. Background

A. Static Random-Access Memory (SRAM)

Fig. 2 illustrates the standard six-transistor (6T) SRAM cell consisting of two cross-coupled inverters $(M_1\&M_2, M_3\&M_4)$ and two access transistors (M_5, M_6). The data bit is differentially stored at nodes Q and $\overline{Q}$. To connect a memory cell to the differential BL and Bit-Line-Bar (BLB) (which are shared among multiple cells in an array), the WL voltage V_{WL} is employed. In typical computing systems, memory cells are organized in arrays, as exemplified in Fig. 2, depicting an array of N words, each consisting of four SRAM cells. A standard SRAM cell has two basic operations:

- *Read*: Firstly, both BLs are pre-charged to the supply voltage V_{DD}. Then, the address decoder activates the cell by driving the WL to V_{DD}. As either Q or $\overline{Q}$ is 0 V, one of the BLs is discharged via M_2 or M_4 while the complementary BL remains at V_{DD}. This differential output signal is captured by Sense Amplifiers (SAs).
- *Write*: Updating the data stored in the memory cell starts with pre-charging both BLs to V_{DD}. Then, the BLs are driven to the desired voltages by discharging one of them to 0 V via the write circuitry. Finally, activating the corresponding WL lets the BL voltage propagate to the memory cell. Given proper transistor dimensions, the BL overwrites the cell data with the desired write data.

The operational principle of SRAM is heavily dependent on analog effects, which opens up creative possibilities for using the circuits discussed in this section for in-memory operations.

B. Discharge-Based IMC

This paradigm relies on operating SRAM cells off-spec and is also referred to as current-domain in-SRAM computing [17]. The fundamental idea involves storing one operand in the memory cell while applying other operand via the WL. As shown in Fig. 3, this can be used to implement multiplications. Firstly, weight (in this case, "1") is stored in the memory cell, and the BLs are pre-charged to V_{DD}. Next, input data is applied to the WL. If weight d is "0", the BL remains at V_{DD}. However, if it is "1", the WL voltage results in a discharge on BLB which is proportional to the product of operands:

$$\delta V(t) \propto V_{WL} \cdot d \cdot t \tag{1}$$

In the simplest case, the WL voltage is binary so that readout can be achieved with adapted SAs. However, this principle can be extended to multiple-bit computations with two key ideas:

1. Quantizing the WL voltage with a Digital-to-Analog Converter (DAC) [8]. The discharge is proportional to this voltage and captured with an Analog-to-Digital Converter (ADC) after a constant sampling time to obtain a product of the cell's data and the applied WL voltage.

2. Interpreting the data of multiple memory cells as a bit vector representing an N-bit integer. This requires a mechanism to represent bit weights which can be achieved in the time, charge, or current domain with reasonable overhead [18]. The weighted discharges are then combined, and the overall discharge is sampled with an ADC.

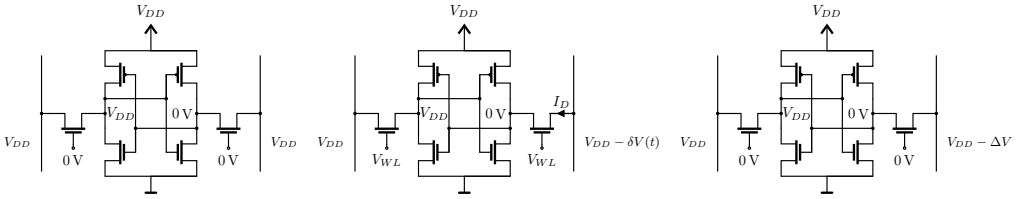
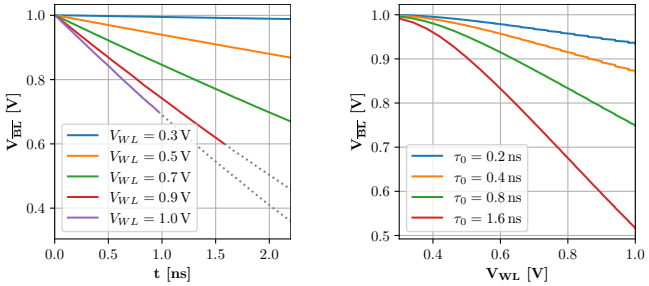


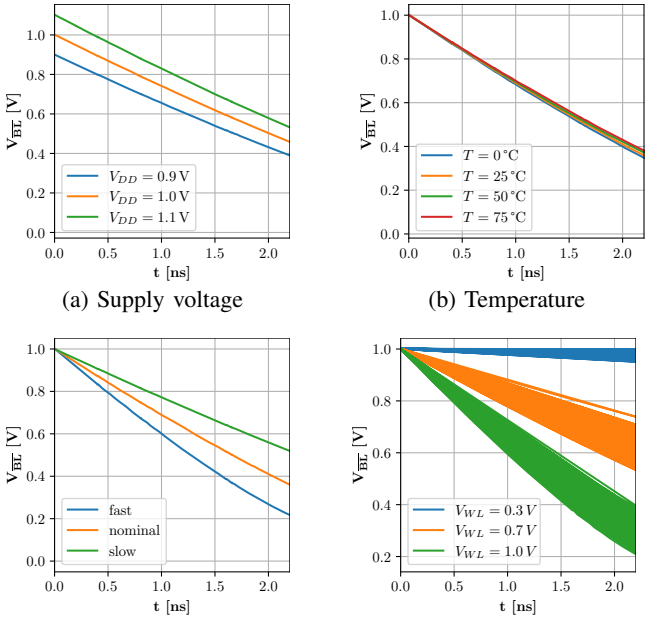
Fig. 3: Multiplication sequence: holding "1" with pre-charged BLs (left), discharge with driven WLs (center), final result (right)



(a) BLB over time with dotted curves indicating saturation

(b) WL voltage dependency when sampled at $t = \tau_0$

Fig. 4: BLB discharge non-idealities.



(a) Supply voltage

(b) Temperature

(c) Process corners

(d) Mismatch (1000 samples)

Fig. 5: Influence of PVT variations on the BLB discharge in TSMC 65 nm technology

A synthesis of these ideas enables multiplications of two multi-bit words [8]. However, these in-SRAM multiplications are currently limited to small bit widths (1-5 bits).

III. In-SRAM Computing Error Sources

To understand why practical implementations of in-SRAM multiplication circuits are limited to small bit widths, we analyze the effects of non-idealities and operating condition variations at the circuit level. Even though the error sources are presented independently in this section, they all co-occurall of them occur simultaneously in any discharge-based IMC circuit. The data in this section is based on circuit simulation for a TSMC 65 nm Complementary Metal-Oxide Semiconductor (CMOS) technology. Further error sources with subordinate impact are analyzed in [19].

1) Circuit Nonlinearity: In-6T-SRAM multipliers operate based on a data-dependent current through the pass transistor. When the stored data is zero, there is no discharge, and the BLB voltage remains V_{DD}. However, if the data is "1" and a voltage representing "0" is applied via the WL, a small discharge occurs due to the non-zero source-drain current of the Metal Oxide Semiconductor Field Effect Transistor (MOSFET) at V_{th} (see Fig. 4a). This asymmetry can lead to different results for $a \times b$ and $b \times a$, degrading multiplier's accuracy.

Moreover, the quadratic current-voltage relationship of MOSFETs introduces a nonlinear discharge dependency on the applied WL voltage (see Fig. 4b). The quantization performed with a conventional DAC consequently leads to nonlinear multiplication results. The adoption of a nonlinear DAC is a potential solution [15], even though its practical circuit implementation poses significant challenges.

Another non-ideality arises from the transition of pass transistors from the saturation to the linear region with increasing BLB discharges. The saturation condition for M_6 is:

$$V_{\overline{BL}} \geq V_{WL} - V_{th} \tag{2}$$

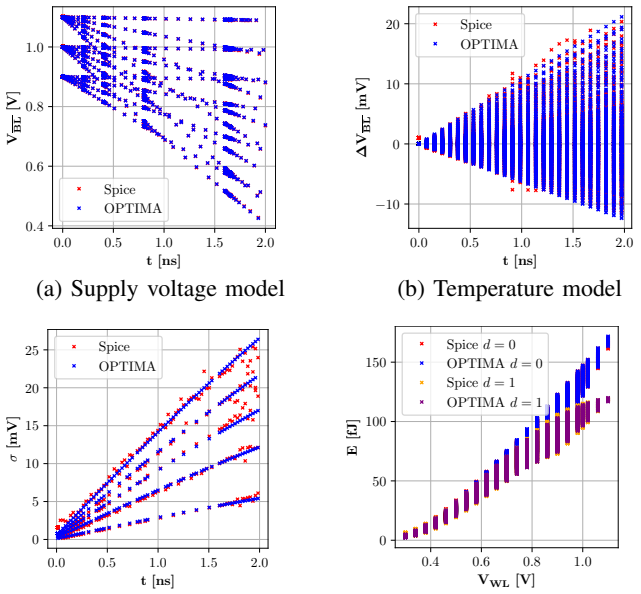
If the BLB discharges below this threshold, the transistor enters the linear region, which leads to a reduced current and therefore slower discharge. Selecting an appropriate ADC sampling time τ_0 ensures that the pass transistors remain in saturation during the BLB discharge. However, smaller τ_0 values lead to reduced BLB voltage swings, degrading the Signal to Noise Ratio (SNR).

2) PVT Variations: PVT variations are the second major limiting factor for accurate in-6T-SRAM computing. Their effect on the discharge voltage is shown in Fig. 5. While temperature fluctuations vary the discharge speed only slightly, supply voltage and process variations alter the discharge voltage curves significantly. Note that supply voltage changes do not only affect the SRAM circuit, but also the thresholds of ADCs and DACs. The variations are also data dependent, e.g. in Fig. 5d, the transistor mismatch-induced deviation grows with the applied WL voltage.

IV. Our Modeling Framework: OPTIMA

Our modeling framework OPTIMA is available open source[1] and facilitates the simulation of analog BL voltage in an event-based fashion, akin to digital simulation tools. This promises significantly shorter runtimes. We accomplished this through the following steps:

[1]The source code is available at https://github.com/sevjaeg/optima.



(a) Supply voltage model (b) Temperature model

(c) Mismatch model (d) Discharge energy model

Fig. 6: OPTIMA discharge modeling evaluation

- Execute thorough multi-corner circuit simulations.
- Develop behavioral models for essential analog metrics, integrating non-idealities and accounting for PVT variations using circuit equations and simulation data.
- Incorporate these models into a versatile discrete-time simulation framework written in System Verilog. Our model is adaptable to a wide range of discharge-based in-SRAM operations.

The behavioral modeling within OPTIMA consists of parameterized discharge and energy models:

A. Discharge Models

To model the BLB discharge accurately, OPTIMA follows an iterative approach. Firstly, the voltage is modeled as a function of time and WL voltage. In the next steps, variation sources are added. All models are based on polynomial functions. In this manuscript, $\mathrm{p}_n(X)$ defines a polynomial of the variable X of degree n with $n+1$ coefficients.

As expressed in Eq. 1, the voltage on the BLB is time-dependent and influenced by the WL voltage. However, as illustrated in Fig. 4, this relationship is nonlinear. Therefore, the function

$$V_{\overline{BL}}\left(t, V_{WL}\right) = V_{DD} + \mathrm{p}_4(V_{\mathrm{od}}) \cdot \mathrm{p}_2(t) \tag{3}$$

with the overdrive voltage $V_{\mathrm{od}} = V_{WL} - V_{th}$ is used to model the BLB discharge. This model already incorporates the nonlinearities as presented in Section III-1.

This model is extended by a supply voltage function

$$V_{\overline{BL}}\left(t, V_{WL}, V_{DD}\right) = V_{\overline{BL}}\left(t, V_{WL}\right) \cdot \mathrm{p}_2(\Delta V_{DD}) \tag{4}$$

with $\Delta V_{DD} = V_{DD} - V_{DD,\mathrm{nom}}$.

As depicted in Fig. 5b, temperature has only a minor effect. Therefore, it is modeled as an additive error term for deviations from the nominal temperature T_{nom} using the function

$$\begin{aligned} V_{\overline{BL}}\left(t, V_{WL}, V_{DD}, T\right) =& V_{\overline{BL}}\left(t, V_{WL}, V_{DD}\right) \\ &+ \left(t \cdot \left(T - T_{\mathrm{nom}}\right) \cdot \mathrm{p}_3(V_{WL})\right) \end{aligned} \tag{5}$$

In contrast to the parameters discussed so far, process variations are an inherently stochastic property. Therefore, they are modeled with a statistical approach. As mismatch causes Gaussian variations in the BLB voltage, its standard deviations σ are modeled as:

$$\sigma\left(t, V_{WL}\right) = \mathrm{p}_3(t) \cdot \mathrm{p}_3(V_{WL}) \tag{6}$$

B. Energy Models

To assess energy-accuracy trade-offs in SRAM IMC, the energy consumption of the cells has to be modeled as well. It is mainly related to charging the BL capacitances during the pre-charge phases.

The energy consumed in writes is data-independent due to the symmetric cell layout and can thus be modeled as:

$$E_{\mathrm{wr}}\left(V_{DD}, T\right) = \mathrm{p}_2(V_{DD}) \cdot \mathrm{p}_1(T) \tag{7}$$

In contrast, the discharge energy depends on BLB discharge which depends on both operands (data d and WL voltage):

$$E_{\mathrm{dc}}\left(d, V_{DD}, V_{WL}, T\right) = \mathrm{p}_1(V_{DD}) \cdot \mathrm{p}_3(\Delta V_{\overline{BL}}) \cdot \mathrm{p}_1(T) \tag{8}$$

The BLB discharge $\Delta V_{\overline{BL}}$ depends on d, V_{DD}, V_{WL}, and T and is calculated with the models presented in Eq. 3–5.

C. Model Evaluation

Least-squares fitting is employed to determine the coefficients for the models in Eq. 3–8 based on extensive simulation data. These parameters are subsequently incorporated into the discrete-time simulation model. For the transistor mismatch, the Gaussian distribution with σ from Eq. 6 is sampled for each discharge. The resulting voltages and energies are illustrated in Fig. 6. The Root Mean Square (RMS) modeling errors are 0.76 mV (basic discharge), 0.88 mV (VDD), 0.76 mV (temperature), 0.59 mV (mismatch σ), 0.15 fJ (write energy), and 0.74 fJ (discharge energy) respectively. These values are below typical ADC Least Significant Bit (LSB) voltages for in-SRAM operations, indicating a sufficient accuracy for reliable analyses of in-SRAM computing circuits.

V. Case Study: In-SRAM Multiplier

We showcase the application of OPTIMA using the 4-bit multiplication circuit presented in [8]. This circuit employs the discharge principle illustrated in Fig. 3 applied to a 4-bit per word array, as depicted in Fig. 2. The WLs voltages are controlled using a 4-bit DAC, and the discharge occurs at intervals of τ_0, $2\tau_0$, $4\tau_0$, and $8\tau_0$ on the different BLBs to implement bit weighing. Subsequently, the discharge voltages are sampled using switches and capacitors, and the combined discharge voltage is captured using an ADC. For simplicity,

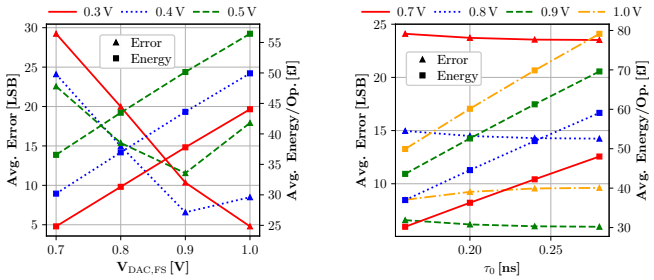


Fig. 7: Design space corners simulations with OPTIMA for different values of $V_{\mathrm{DAC},0}$ with $\tau_0 = 1.6\,\mathrm{s}$ (left) and $V_{\mathrm{DAC,FS}}$ with $V_{\mathrm{DAC},0} = 0.4\,\mathrm{V}$ (right).

we omit the analog accumulation step from the original publication and concentrate on the multiplication process, which can be modeled efficiently with OPTIMA.

To demonstrate our methodology, we establish a design space characterized by three circuit parameters:

- τ_0: Discharge time of the least significant BLB
- $V_{\mathrm{DAC},0}$: Output voltage of the DAC for data word "0"
- $V_{\mathrm{DAC,FS}}$: Full-scale output voltage of the DAC

We select 48 design corners and simulate the circuit using the OPTIMA framework. The results of this design-space exploration are depicted in Fig. 7. A higher value of $V_{\mathrm{DAC,FS}}$ results in a linear increase in energy consumption but is associated with higher accuracies in most cases. Increasing $V_{\mathrm{DAC},0}$ or τ_0 also leads to higher energy consumption. The former has a positive impact on multiplication errors, while the latter has minimal influence on accuracy.

We select three interesting configurations to perform PVT analyses with OPTIMA. The first corner *fom* is selected based on maximizing a Figure of Merit (FOM) combining the averages of error after quantization ϵ_{mul} and energy per operation E_{mul}:

$$\mathrm{FOM} = \frac{1}{\overline{\epsilon_{\mathrm{mul}}} \cdot \overline{E_{\mathrm{mul}}}} \tag{9}$$

The second corner *power* is the one with the minimum energy per multiplication and the third corner *mismatch* shows the smallest standard deviation at the maximum discharge (i.e., it is least impacted by process variation). Table I summarizes the corresponding parameters.

The PVT analysis results, including sampling mismatch corners, are presented in Fig. 8. Deviations in average multiplication results signify the impact of circuit nonlinearities, while high standard deviations indicate susceptibility to mismatch. In terms of the *power* configuration, issues are observed in both cases, with the *variation* corner performing notably worse than *fom* for small values. However, for large values, it demonstrates robustness against process variation. Voltage and temperature fluctuations also exert a significant effect on the error level, with the *fom* corner proving to be the least susceptible to these variations.

Drawing conclusions from our design-space exploration using OPTIMA, we determine that the *fom* configuration produces the most favorable results. At an operating frequency of 167 MHz, it exhibits an average multiplication

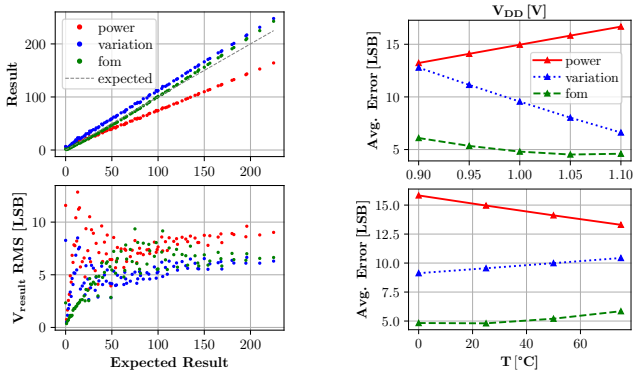


Fig. 8: Average multiplication results and analog standard deviations (left) as well as influence of voltage and temperature variations on the error (right) for the selected corners.

error of 4.8 LSBs. The worst-case analog standard deviation is 5.04 mV. For a single operation, including write and multiplication, the average energy consumption is 1.05 pJ. For the multiplication circuit, OPTIMA achieves a speedup of $101\times$ for iteration over the input space and design corners and $28.1\times$ for mismatch Monte Carlo (MC) sampling compared to circuit simulation in Cadence Virtuoso.

TABLE I: Selected design corners

Corner	τ_0	$V_{\mathrm{DAC},0}$	$V_{\mathrm{DAC,FS}}$	$\overline{\epsilon_{\mathrm{mul}}}$	$\overline{E_{\mathrm{mul}}}$
fom	0.16 ns	0.3 V	1.0 V	4.78	44 fJ
power	0.16 ns	0.3 V	0.7 V	15	37 fJ
variation	0.24 ns	0.4 V	1.0 V	9.6	69.8 fJ

VI. Application Analysis

In this research, we assess the effectiveness of the proposed in-SRAM multiplier configurations for DNN inference. To accomplish this, we use these multipliers in deep learning models for image classification, specifically VGG16, VGG19 [20], ResNet50, and ResNet101 [21], which are trained on the ImageNet dataset [22]. Furthermore, we apply these models to the CIFAR-10 [23] dataset to assess their performance.

In the initial series of experiments, pre-trained DNNs obtained from the Keras model zoo, originally trained on the ImageNet dataset, are employed to showcase the efficacy of the selected in-memory multiplier configurations. This pre-trained DNNs utilized a FLOAT32 number representation. These experiments aim to utilize our proposed multiplier for all multiplication operations within these DNNs. To achieve this, we quantized the pre-trained DNNs to an INT4 number representation using post-training quantization.

This quantization process adhered to the specifications of TensorFlow Lite, with INT8 being replaced by INT4 along with corresponding adjustments to the range, restrictions, and other specifications. Subsequently, retraining procedures are implemented to mitigate the impact of quantization on the relevant metrics. The proposed in-memory multiplier configurations are then applied to execute all multiplication operations within the DNNs.

Given the application's emphasis on image classification, the evaluation focuses on top-1 and top-5 accuracies as key metrics. The results, depicting the utilization of the in-memory

TABLE II: DNN classification accuracies for ImageNet

Model	Number of Multiplications [$\times 10^9$]	Baseline FLOAT32		Baseline INT4		In-Memory *fom* (INT4)		In-Memory *power* (INT4)		In-Memory *variation* (INT4)	
		Top-1 Accuracy [%]	Top-5 Accuracy [%]	Top-1 Accuracy [%]	Top-5 Accuracy [%]	Top-1 Accuracy [%]	Top-5 Accuracy [%]	Top-1 Accuracy [%]	Top-5 Accuracy [%]	Top-1 Accuracy [%]	Top-5 Accuracy [%]
VGG16	15.61	70.30	90.10	69.25	89.62	68.97	89.11	64.45	81.79	38.22	47.81
VGG19	19.77	71.30	90.00	70.09	89.78	69.91	89.24	63.34	79.61	36.66	48.37
ResNet50	4.14	74.90	92.10	73.48	91.75	73.39	91.65	61.56	80.88	48.07	56.71
ResNet101	7.87	76.40	92.80	75.12	91.91	74.95	91.63	59.77	78.49	48.45	53.19

TABLE III: DNN classification accuracies for CIFAR10

Model	FLOAT32	INT4	In-Memory *fom* (INT4)	In-Memory *power* (INT4)	In-Memory *variation* (INT4)
	Top-1 Accuracy [%]	Top-1 Accuracy [%]	Top-1 Accuracy [%]	Top-1 Accuracy [%]	Top-1 Accuracy [%]
VGG16	92.24	92.04	91.98	87.39	68.10
VGG19	92.71	92.42	92.29	89.79	66.85
ResNet50	93.10	92.86	92.83	90.81	73.83
ResNet101	93.35	93.06	93.04	90.42	69.77

multiplier in three configurations for the quantized ImageNet-trained DNNs, are presented in Table II.

To further evaluate our findings, a second set of experiments is conducted on DNNs to classify the CIFAR-10 dataset. Again, the DNNs is obtained by employing INT4 quantization. Additionally, the last layer is replaced with a fully-connected layer containing 10 neurons (reflecting the number of classes in CIFAR-10), and transfer learning is utilized for training. The same experimental conditions as before are applied to assess the performance of the multiplier configurations. The multiplication operations in each DNN closely aligned with the values in Table II, as the modifications were confined to the last layer, resulting in minimal changes ($< 0.03\%$) in the number of multiplication for an individual inference.

Among the selected multiplier configurations, the *fom* corner is most effective in our application. It exhibits a marginal decrease in accuracy compared to quantized INT4 DNNs (considered as baseline) and outperforms all other configurations across all DNNs. For ImageNet dataset, the top-1 accuracies degrade by a mere 1.42% and 0.18% and for the CIFAR-10 dataset, by 0.315% and 0.06% compared to FLOAT32 and INT4 representations. The *power* configuration trades off accuracy for reduced energy consumption and shows more significant accuracy reductions. Interestingly, *variation* corner achieves only top-1 accuracies of 42.85% and 69.63% for ImageNet and CIFAR-10, respectively on average. The reason for this is its high error level for multiplications with small operands (see Fig. 8), which dominate in DNN workloads.

VII. Conclusion

To overcome the challenges of selecting one design for in-SRAM computing circuits, we have introduced OPTIMA. It is a fast yet accurate design-space exploration technique exhibiting $\sim$100 $\times$ simulation speed-up while maintaining an RMS modeling error of 0.88 mV for an in-SRAM multiplier. Our techniques' exploration yields an optimized multiplier with 1.05 pJ energy consumption per 4-bit operation, and classification accuracies of 71.8% (top-1) and 90.4% (top-5) for ImageNet and 92.5% for CIFAR-10 datasets, respectively, when applied in quantized DNNs. We made our OPTIMAptima tool flow available as open source [https://github.com/sevjaeg/optima].

References

[1] K. Rupp. 50 years of microprocessor trend data, 2022.

[2] W. A. Wulf and S. A. McKee. Hitting the memory wall. *ACM SIGARCH Computer Architecture News*, 23(1):20–24, 1995.

[3] M. Horowitz. 1.1 Computing's energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pp. 10–14, 2014.

[4] M. Ferdman *et al.* Clearing the clouds: A study of emerging scale-out workloads on modern hardware. In *Proceedings of the Seventeenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS XVII, pp. 37–48, 2012.

[5] A. Boroumand *et al.* Google workloads for consumer devices: Mitigating data movement bottlenecks. *SIGPLAN Not.*, 53(2):316–331, 2018.

[6] A. Gebregiorgis *et al.* A survey on memory-centric computer architectures. *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, 18(4):1–50, 2022.

[7] Y. Yu and N. K. Jha. Energy-efficient monolithic three-dimensional on-chip memory architectures. *IEEE Transactions on Nanotechnology*, 17(4):620–633, 2017.

[8] M. Ali *et al.* IMAC: In-memory multi-bit multiplication and ACcumulation in 6T SRAM array. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 67(8):2521–2531, 2020.

[9] S. Seyedfaraji *et al.* SMART: Investigating the impact of threshold voltage suppression in an in-SRAM multiplication/accumulation accelerator for accuracy improvement in 65 nm CMOS technology. In *2022 25th Euromicro Conference on Digital System Design (DSD)*, pp. 821–826. IEEE, 2022.

[10] S. Aga *et al.* Compute caches. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 481–492, 2017.

[11] J. Wang *et al.* A 28-nm compute SRAM with bit-serial logic/arithmetic operations for programmable in-memory vector computing. *IEEE Journal of Solid-State Circuits*, 55(1):76–86, 2020.

[12] C. Eckert *et al.* Neural cache: Bit-serial in-cache acceleration of deep neural networks. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018.

[13] M. Kang *et al.* A multi-functional in-memory inference processor using a standard 6T SRAM array. *IEEE Journal of Solid-State Circuits*, 53(2):642–655, 2018.

[14] K. Sanni *et al.* A charge-based architecture for energy-efficient vector-vector multiplication in 65nm CMOS. In *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–5. IEEE, 2018.

[15] S. Seyedfaraji *et al.* AID: Accuracy improvement of analog discharge-based in-SRAM multiplication accelerator. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2022.

[16] M. Gong *et al.* A 65nm thermometer-encoded time/charge-based compute-in-memory neural network accelerator at 0.735 pJ/MAC and 0.41 pJ/update. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 68(4):1408–1412, 2020.

[17] Z. Chen *et al.* CAP-RAM: A charge-domain in-memory computing 6T-SRAM for accurate and precision-programmable CNN inference. *IEEE Journal of Solid-State Circuits*, 56(6):1924–1935, 2021.

[18] Z. Lin *et al.* A review on SRAM-based computing in-memory: Circuits, functions, and applications. *Journal of Semiconductors*, 43(3):031401, 2022.

[19] A. Kneip and D. Bol. Impact of analog non-idealities on the design space of 6T-SRAM current-domain dot-product operators for in-memory computing. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 68(5):1931–1944, 2021.

[20] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[21] K. He *et al.* Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.

[22] J. Deng *et al.* Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 248–255, 2009.

[23] A. Krizhevsky. Learning multiple layers of features from tiny images. *University of Toronto*, 2012.